

# A.

## Guest 1

```
sudo apt update -y
```

此指令會更新 Ubuntu 套件管理器的套件清單。`sudo` 代表以管理員身份執行，`-y` 則表示自動同意所有提示，不需手動輸入確認。

```
sudo apt install -y vim qemu-kvm nfs-kernel-server net-tools
```

這行指令用來安裝必要的工具。`vim` 是文字編輯器；`qemu-kvm` 是啟用 KVM 的虛擬化支援；`nfs-kernel-server` 是 NFS 伺服器，用於提供磁碟共享；`net-tools` 則包含如 `ifconfig` 等傳統網路工具。

```
ip a
```

顯示所有網路介面的詳細資訊，包括 IP 位址。這步驟用來確認 `guest1` 的 IP 位址，以便後續 `guest2` 掛載 NFS 使用。

```
qemu-img create -f qcow2 ubuntu.qcow2 10G
```

建立一個 QEMU 的虛擬磁碟映像檔。`-f qcow2` 指定格式為 `qcow2`，支援快照與壓縮；`ubuntu.qcow2` 是檔名；`10G` 是磁碟大小。

```
sudo mkdir -p /mnt/nfs
```

建立一個目錄作為 NFS 共享資料夾，位於 `/mnt/nfs`。`-p` 代表如果上層目錄不存在也會一併建立。

```
sudo chown nobody:nogroup /mnt/nfs
```

更改該資料夾的擁有者與群組為 `nobody` 和 `nogroup`，這是 NFS 預設的匿名帳號。

```
sudo chmod 777 /mnt/nfs
```

設定該資料夾權限為所有人皆可讀寫執行（`rwX`），方便測試使用。

```
mv ubuntu.qcow2 /mnt/nfs
```

將剛剛建立的虛擬磁碟檔案移動到 NFS 分享資料夾中，讓其他 VM 可透過 NFS 使用它。

```
sudo vim /etc/exports
```

使用 `vim` 編輯 `/etc/exports` 設定檔，此檔案用來設定哪些資料夾可透過 NFS 匯出分享。

在檔案末尾加入一行：

```
/mnt/nfs *(rw,sync,no_subtree_check,no_root_squash)
```

`/mnt/nfs` 是被分享的資料夾，`*` 表示所有主機皆可掛載；`rw` 為可讀寫；`sync` 表示寫入立即同步；`no_subtree_check` 提高效能；`no_root_squash` 保留 `root` 權限（測試環境中可接受）。

```
sudo exportfs -arv
```

將上述 NFS 設定匯出並啟用。`-a` 表示全部匯出，`-r` 表示重新匯出，`-v` 顯示詳細資訊。

```
sudo qemu-system-x86_64 \
```

```
-cpu host -enable-kvm -m 2G -smp 1 \
```

```
-drive if=virtio,format=qcow2,file=/mnt/nfs/ubuntu.qcow2 \
```

```
-monitor telnet:127.0.0.1:5500,server,nowait
```

這條指令啟動一台 QEMU VM。`-cpu host` 表示使用主機的 CPU 模型，`-enable-kvm` 啟用硬體虛擬化，`-m 2G` 配置 2GB 記憶體，`-smp 1` 使用一個 CPU 核心。`-drive if=virtio,format=qcow2,file=...` 指定使用 `virtio` 介面的虛擬硬碟，來源為剛剛透過 NFS 分享的磁碟檔。`-monitor telnet...` 表示開啟 `telnet` 模式的 QEMU monitor，用於後續 `live migration` 操作。

---

## Guest 2

```
sudo apt update -y
```

更新本機的套件清單，以取得最新版本。

```
sudo apt install -y qemu-kvm net-tools nfs-common
```

安裝必要套件。`qemu-kvm` 為 QEMU 虛擬機與 KVM 支援；`net-tools` 提供傳統網路工具；`nfs-common` 為 NFS 用戶端工具，讓此 VM 可以掛載 NFS 資

料夾。

`ip a`

顯示本機的網路資訊，以確認 IP 位址與網路連線狀態。

`sudo mkdir -p /mnt/nfs`

建立一個資料夾作為掛載點，用來連接 `guest1` 分享的磁碟。

`sudo mount -t nfs <guest1_ip>:/mnt/nfs /mnt/nfs`

掛載 `guest1` 分享出來的 NFS 資料夾。`<guest1_ip>` 需替換為實際的 `guest1` IP 位址；`-t nfs` 指定檔案系統類型；前者是遠端來源，後者是本地掛載位置。

`sudo qemu-system-x86_64 \`

`-cpu host -enable-kvm -m 2G -smp 1 \`

`-drive if=virtio,format=qcow2,file=/mnt/nfs/ubuntu.qcow2 \`

`-incoming tcp:0:4400`

啟動第二台 VM 並準備接受 live migration。`-cpu host` 和 `-enable-kvm` 提供硬體虛擬化支援；`-m 2G` 和 `-smp 1` 配置資源；`-drive` 指定使用透過 NFS 掛載的虛擬硬碟；`-incoming tcp:0:4400` 表示此 VM 將等待來自來源 VM（`guest1`）的遷移資料，監聽本地 4400 port。

## B.

## Without:

```
m114064548@m114064548-VMware-Virtual-Platform:~$ sudo /usr/lib/linux-tools/6.8.0-85-generic/perf stat timeout 10s sha256sum /dev/zero
Performance counter stats for 'timeout 10s sha256sum /dev/zero':

          9,831.58 msec task-clock                #    0.983 CPUs utilized
             2,267      context-switches         #   230.583 /sec
                 3      cpu-migrations            #    0.305 /sec
             289      page-faults                 #   29.395 /sec
        34,330,911,255      cycles                  #    3.492 GHz                    (60.01%)
                 0      stalled-cycles-frontend   #    0.000 /sec                    (60.01%)
                 0      instructions              #    0.000 insns per cycle         (60.02%)
                 0      branches                   #    0.000 /sec                    (59.98%)
                 0      branch-misses              #    0.000 /sec                    (59.99%)

10.002787393 seconds time elapsed

 8.852975000 seconds user
 0.964800000 seconds sys
```

With:

```
m114064548@m114064548-VMware-Virtual-Platform: $ sudo /usr/lib/linux-tools/6.8.0-85-generic/perf stat timeout 10s sha256sum /dev/zero

Performance counter stats for 'timeout 10s sha256sum /dev/zero':

    9,981.48 msec task-clock                #    0.998 CPUs utilized
      398      context-switches            #   39.874 /sec
        0      cpu-migrations              #    0.000 /sec
      290      page-faults                 #   29.054 /sec
 35,073,362,220 cycles                     #    3.514 GHz                    (60.01%)
        0      stalled-cycles-frontend    #    0.000 /sec                    (60.03%)
        0      instructions                #    0.00 insn per cycle          (59.97%)
        0      branches                   #    0.000 /sec                    (60.01%)
        0      branch-misses               #    0.000 /sec                    (59.98%)

10.003784481 seconds time elapsed

 9.158046000 seconds user
 0.820265000 seconds sys
```

未啟用 KVM 時，CPU 使用率約 0.98，上下文切換 (context-switches/sec) 很高（約 230 次/秒），代表大量系統層級干預，cpu-migrations 也會出現，說明資源調度較多，導致效能偏低。用戶態 (user time) 約 8.85 秒，核心態 (sys time) 約 0.96 秒，執行效能有限。

啟用 KVM 之後，CPU 使用率提升到 0.998，幾乎能完全發揮 CPU 資源，上下文切換大幅下降（只剩 39 次/秒），cpu-migrations 直接為 0，代表 VM 能直接透過硬體虛擬化執行，減少額外系統開銷。用戶態 (user time) 提升到約 9.15 秒，系統態 (sys time) 降到約 0.82 秒，計算更穩定高效。

效能改善原因是，KVM 讓虛擬機直接利用 CPU 虛擬化指令，減少 QEMU 軟體模擬所需的上下文切換、資源調度與系統干預，近乎原生硬體效率。

## C. without

```
m114064548-2@m114064548-2-VMware-Virtual-Platform:~$ iperf3 -c 192.168.232.132
Connecting to host 192.168.232.132, port 5201
[ 5] local 192.168.232.128 port 58200 connected to 192.168.232.132 port 5201
[ ID] Interval           Transfer     Bitrate        Retr  Cwnd
[ 5]  0.00-1.00      sec   410 MBytes  3.44 Gbits/sec    58   1.56 MBytes
[ 5]  1.00-2.00      sec   409 MBytes  3.43 Gbits/sec     0   1.74 MBytes
[ 5]  2.00-3.00      sec   406 MBytes  3.41 Gbits/sec    37   1.35 MBytes
[ 5]  3.00-4.00      sec   402 MBytes  3.38 Gbits/sec     0   1.56 MBytes
[ 5]  4.00-5.00      sec   407 MBytes  3.41 Gbits/sec     0   1.74 MBytes
[ 5]  5.00-6.00      sec   441 MBytes  3.70 Gbits/sec    37   1.38 MBytes
[ 5]  6.00-7.00      sec   447 MBytes  3.75 Gbits/sec     0   1.60 MBytes
[ 5]  7.00-8.00      sec   460 MBytes  3.86 Gbits/sec     0   1.80 MBytes
[ 5]  8.00-9.00      sec   461 MBytes  3.87 Gbits/sec    36   1.47 MBytes
[ 5]  9.00-10.00     sec   458 MBytes  3.84 Gbits/sec     0   1.68 MBytes

- - - - -
[ ID] Interval           Transfer     Bitrate        Retr
[ 5]  0.00-10.00      sec   4.20 GBytes  3.61 Gbits/sec   168
[ 5]  0.00-10.00      sec   4.20 GBytes  3.61 Gbits/sec

iperf Done.
```

## with

```

m114064548-2@m114064548-2-VMware-Virtual-Platform:~$ iperf3 -c 192.168.232.132
Connecting to host 192.168.232.132, port 5201
[ 5] local 192.168.232.128 port 42092 connected to 192.168.232.132 port 5201
[ ID] Interval           Transfer     Bitrate      Retr  Cwnd
[ 5]  0.00-1.00      sec   418 MBytes  3.50 Gbits/sec  113  1.59 MBytes
[ 5]  1.00-2.00      sec   428 MBytes  3.59 Gbits/sec   0  1.77 MBytes
[ 5]  2.00-3.00      sec   445 MBytes  3.74 Gbits/sec   2  1.43 MBytes
[ 5]  3.00-4.00      sec   450 MBytes  3.78 Gbits/sec   0  1.65 MBytes
[ 5]  4.00-5.00      sec   444 MBytes  3.72 Gbits/sec   0  1.83 MBytes
[ 5]  5.00-6.00      sec   443 MBytes  3.71 Gbits/sec  12  1.51 MBytes
[ 5]  6.00-7.00      sec   457 MBytes  3.83 Gbits/sec   0  1.72 MBytes
[ 5]  7.00-8.00      sec   458 MBytes  3.84 Gbits/sec   1  1.35 MBytes
[ 5]  8.00-9.00      sec   460 MBytes  3.86 Gbits/sec   0  1.59 MBytes
[ 5]  9.00-10.00     sec   462 MBytes  3.87 Gbits/sec   0  1.79 MBytes
-----
[ ID] Interval           Transfer     Bitrate      Retr
[ 5]  0.00-10.00      sec  4.36 GBytes  3.74 Gbits/sec  128
[ 5]  0.00-10.01      sec  4.36 GBytes  3.74 Gbits/sec
iperf Done.

```

第一張圖顯示 10 秒內，平均吞吐量約為 3.74 Gbits/sec，重傳次數 128 次，傳輸總量約 4.36 GBytes，這是「啟用 virtio」的效能。

第二張圖則顯示平均吞吐量為 3.61 Gbits/sec，重傳次數 168 次，傳輸總量約 4.20 GBytes，明顯較低、重傳次數較多，這對應「未啟用 virtio」的狀況。

效能上的差異主要來自於 virtio 是為虛擬化環境設計的 paravirtualized 網路驅動，能大幅減少主機 hypervisor 與 VM 之間的資料複製，以及 CPU 資源損耗。所以啟用 virtio 可以讓網路效能更高、延遲更低、丟包重傳也較少，並且吞吐量與穩定性都明顯優於傳統網卡。

## D.

前兩題是在 perf and iperf during the live migration is progressing 的狀態下，在 VM 進行 live migration 時，同步執行 perf 和 iperf 測試會發現以下效能現象：

perf 測試的 CPU 使用率、上下文切換等指標會在遷移過程中短暫波動，尤其在複製記憶體或切換階段可能出現 CPU 使用率降低、上下文切換次數增加現象，代表系統負載轉移與暫時未能穩定。

iperf 網路測試則會發現，網路吞吐量 (bitrate) 會在 live migration 剎那短暫下降或波動，出現重傳數 (Retr) 上升、延遲增加，等 migration 結束後才恢復正常穩定。

## E.

- 首先，系統會先進行記憶體預複製，把 VM 的大部分記憶體內容複製到目的主機。在這個階段，VM 仍然在原本的主機上運作。接著，進入持續同步的階段，系統會不斷追蹤並同步在 VM 運行期間被修改過的記憶體頁面。當需要同步的資料量非常少時，系統會短暫停機，這個時間非常短，通常只有幾毫秒，並將最後的差異數據傳送過去。最後，所有資料都同步完畢，VM 在目的主機上正式繼續執行，來源主機則釋放所有資源。
- 為什麼需要 Live Migration？主要就是為了確保服務不中斷，當你需要對主機硬體或系統進行維護、升級時，不用再讓服務停機，可以直接把 VM 移走。當某台主機負載過高時，可以把一些 VM 移到其他較空閒的主機上來平衡負載，維持系統穩定。在離峰時段，也可以把多個 VM 集中到少數幾台主機上，讓其他主機可以關機節能。

## F.

### 1. What is fault-tolerance in virtualization technology and why do we require it?

容錯（Fault-tolerance）是指系統在部分硬體或軟體發生故障時，依然能保持正常運作且不中斷服務的能力。在虛擬化技術中，容錯通常透過多台實體主機間的虛擬機狀態同步，當主機發生故障時，備援主機能迅速接管並繼續執行虛擬機，避免服務中斷。這對企業及重要服務系統極為重要，因為系統故障會導致資料遺失、服務不可用，甚至造成巨大經濟損失與信譽受損。

---

### 2. What are the relationships between live migration and fault-tolerance?

實時遷移（Live Migration）是一種技術，允許在虛擬機持續運行的狀態下，將其從一台實體主機遷移到另一台，並且對使用者幾乎沒有中斷感知。容錯機制和實時遷移關係密切，因為容錯通常依賴類似的狀態同步技術來保持虛擬機的

最新狀態。在發生故障時，備援主機可以利用先前同步的虛擬機狀態快速接管，達到無縫切換的效果。